

# Implementation Plan: Vector Dataset Loader & Brute-Force kNN

**Deadline: September 7th, 2026**

This plan covers the foundational tasks for the vector search engine: reading the benchmark datasets and establishing a ground-truth baseline using exact brute-force search.

## Section 1: Data Structure Skeleton

Before loading data, we need flexible, robust structures to hold it. This requires modern C++ design to accommodate both `float` (for vectors) and `int32_t` (for ground-truth indices) seamlessly.

### Implementation Steps:

1. **Define Metadata:** Create a struct `Metadata` containing an `std::unordered_map<std::string, std::any>`. This allows arbitrary payload data (like document IDs or tags) to be attached to vectors later.
2. **Define VectorRecord:** Create a templated struct `VectorRecord<T>`.
  - It should contain an `int id` (an internal sequential identifier), an `std::vector<T>` `vector` (the actual data), and an instance of `Metadata`.
  - Add a helper method `size_t dimension() const` that returns `vector.size()`.
3. **Setup Project:** Initialize a CMake project. Ensure it compiles with C++17 (minimum) or C++20, as `std::any` and templated abstractions are required.

## Section 2: Binary Dataset Loader ( `.fvecs` / `.ivecs` )

The SIFT datasets use a highly specific binary format. Every vector in the file is prefixed by a 4-byte little-endian integer ( `d` ) specifying its dimension, immediately followed by `d` elements of 4-bytes each (either `float` or `int32_t`).

### Implementation Steps:

1. **Create VectorStoreIO:** Implement a class with a static templated method: `static std::vector<VectorRecord<T>> read_vecs(const std::string& file_path)`.
2. **Binary File Handling:**
  - Open the file using `std::ifstream` with the `std::ios::binary` flag.
  - Implement strict error handling: throw an `std::runtime_error` if the file fails to open. Rely on RAII (the `ifstream` destructor) to ensure the file handle closes safely.
3. **The Parsing Loop:**
  - Read the first 4 bytes into an `int32_t` variable ( `dim` ) using `file.read(reinterpret_cast<char*>(&dim), sizeof(dim))`.
  - If the read succeeds, instantiate a `VectorRecord<T>`, set its `id`, and resize its inner vector to `dim`.

- Read the next `dim * sizeof(T)` bytes directly into the vector's underlying buffer:  
`file.read(reinterpret_cast<char*>(record.vector.data()), dim * sizeof(T))` .
  - Loop this process until `file.read` evaluates to false (EOF).
4. **Validation Checkpoint:** Download the SIFT-small dataset. Write a test asserting that `read_vecs<float>("siftsmall_base.fvecs")` returns exactly 10,000 vectors, and each has a dimension of 128.

## Section 3: Brute-Force kNN ( `flat_search` )

This section implements exact Nearest Neighbor search via a linear scan. The goal is to compare a query vector against every base vector, keeping track of the top-k closest matches.

### Implementation Steps:

#### 1. Distance Metric (Squared L2):

- Write a function to calculate the Euclidean distance between two `std::vector<float>` of the same size.
- *Crucial Optimization:* Do **not** apply the final `std::sqrt()` . Minimizing the *squared* L2 distance (  $x^2 + y^2$  ) yields the exact same ranking as true L2, but saves massive CPU overhead.

#### 2. Top-k Selection via Priority Queue:

- To avoid sorting the entire dataset of distances, use a max-heap:  
`std::priority_queue<std::pair<float, int>>` . The pair holds (distance, vector\_id) .
- By default, C++ `priority_queue` puts the *largest* element at the top. This is perfect for our bounded queue.

#### 3. The `flat_search` Loop:

- Iterate through the entire base dataset. Calculate the squared L2 distance between the query and the current base vector.
- If the queue size is  $< k$ , push the (distance, id) pair.
- If the queue size is exactly  $k$ , compare the calculated distance to the top of the queue ( `pq.top().first` ). If the new distance is *smaller*, `pq.pop()` the top and `pq.push()` the new pair.
- Extract the indices from the queue and reverse them (since popping a max-heap yields largest-to-smallest, but we want closest-to-farthest).

4. **Recall Validation:** Run `flat_search(query, k=100)` for the 100 queries in SIFT-small. Compare your result indices against the ground-truth indices loaded from `siftsmall_groundtruth.ivecs` . Calculate the intersection. Because brute force is exact, your Recall@100 *must* be exactly 1.0 (100%).